

[C](#) [C Basics](#) [C Data Types](#) [C Operators](#) [C Input and Output](#) [C Control Flow](#) [C Functions](#) [C Arrays](#) [C Strin](#)

C typedef

Last Updated : 10 Apr, 2023

The **typedef** is a keyword that is used to provide existing data types with a new name. The C typedef keyword is used to redefine the name of already existing data types.

When names of datatypes become difficult to use in programs, typedef is used with user-defined datatypes, which behave similarly to defining an alias for commands.

C typedef Syntax

```
typedef existing_name alias_name;
```

After this declaration, we can use the *alias_name* as if it were the real *existing_name* in our C program.

Example of typedef in C

```
typedef long long ll;
```

Below is the C program to illustrate how to use typedef.

C

```
// C program to implement typedef
#include <stdio.h>

// defining an alias using typedef
typedef long long ll;

// Driver code
int main()
{
    // using typedef name to declare variable
    ll var = 20;
    printf("%ld", var);
}
```

```
    return 0;  
}
```

Output

20

Use of typedef in C

Following are some common uses of the typedef in C programming:

- The typedef keyword gives a meaningful name to the existing data type which helps other users to understand the program more easily.
- It can be used with structures to increase code readability and we don't have to type struct repeatedly.
- The typedef keyword can also be used with pointers to declare multiple pointers in a single statement.
- It can be used with arrays to declare any number of variables.

1. typedef struct

typedef can also be used with structures in the C programming language. A new data type can be created and used to define the structure variable.

Example 1: Using typedef to define a name for a structure

C

```
// C program to implement  
// typedef with structures  
#include <stdio.h>  
#include <string.h>  
  
// using typedef to define an alias for structure  
typedef struct students {  
    char name[50];  
    char branch[50];  
    int ID_no;  
} stu;  
  
// Driver code
```

```
int main()
{
    stu st;
    strcpy(st.name, "Kamlesh Joshi");
    strcpy(st.branch, "Computer Science And Engineering");
    st.ID_no = 108;

    printf("Name: %s\n", st.name);
    printf("Branch: %s\n", st.branch);
    printf("ID_no: %d\n", st.ID_no);
    return 0;
}
```

Output

```
Name: Kamlesh Joshi
Branch: Computer Science And Engineering
ID_no: 108
```

2. typedef with Pointers

typedef can also be used with pointers as it gives an alias name to the pointers. Typedef is very efficient while declaring multiple pointers in a single statement because pointers bind to the right on the simple declaration.

Example:

```
typedef int* Int_ptr;
Int_ptr var, var1, var2;
```

In the above statement var, var1, and var2 are declared as pointers of type int which helps us to declare multiple numbers of pointers in a single statement.

Example 2: Using typedef to define a name for pointer type.

C

```
// C program to implement
// typedef with pointers
#include <stdio.h>

typedef int* ptr;
```

```
// Driver code
int main()
{
    ptr var;
    *var = 20;

    printf("Value of var is %d", *var);
    return 0;
}
```

Output

Value of var is 20

3. typedef with Array

typedef can also be used with an array to increase their count.

Example:

```
typedef int arr[20]
```

Here, arr is an alias for an array of 20 integer elements.

```
// it's same as Arr[20], two-Arr[20][23];
arr Arr, two-Arr[23];
```

Example 3: Using typedef to define an alias for Array.

C

```
// C program to implement typedef with array
#include <stdio.h>

typedef int Arr[4];

// Driver code
int main()
{
    Arr temp = { 10, 20, 30, 40 };
    printf("typedef using an array\n");
}
```

```
for (int i = 0; i < 4; i++) {  
    printf("%d ", temp[i]);  
}  
return 0;  
}
```

Output

typedef using an array
10 20 30 40

C typedef vs #define

The following are the major [difference between the typedef and #define](#) in C:

1. #define is capable of defining aliases for values as well, for instance, you can define 1 as ONE, 3.14 as PI, etc. Typedef is limited to giving symbolic names to types only.
2. Preprocessors interpret #define statements, while the compiler interprets typedef statements.
3. There should be no semicolon at the end of #define, but a semicolon at the end of typedef.
4. In contrast with #define, typedef will actually define a new type by copying and pasting the definition values.

Below is the C program to implement #define:

C

```
// C program to implement #define  
#include <stdio.h>  
  
// macro definition  
#define LIMIT 3  
  
// Driver code  
int main()  
{  
    for (int i = 0; i < LIMIT; i++) {  
        printf("%d \n", i);  
    }  
    return 0;  
}
```

```
}
```

Output

```
0  
1  
2
```

FAQs on typedef in C

1. What is typedef in C?

The C typedef statement defines an alias or a nickname for the already existing data type.

2. What is typedef struct?

The typedef struct is the statement used to define an alias for the structure data type.

3. What is typedef enum?

The typedef enum is used to define the alias for the enumeration data type.

Three 90 Challenge is back on popular demand! After processing refunds worth INR 1CR+, we are back with the offer if you missed it the first time. Get 90% course fee refund in 90 days. [Avail now!](#)

Summer-time is here and so is the time to skill-up! More than 5,000 learners have now completed their journey from **basics of DSA to advanced level development programs** such as Full-Stack, Backend Development, Data Science.

And why go anywhere else when our [DSA to Development: Coding Guide](#) will help you master all this in a few months! Apply now to our [DSA to](#)